

PoV Multi-Agent

Atendimento coordenado por 5 agentes de IA, com MongoDB Atlas como plano de dados e de coordenação

Guia comercial e técnico + roteiro de apresentação ao cliente

1. O pitch — linguagem de negócio

O problema

Atendimento ao cliente hoje é ou 100% humano (caro, lento, não escala) ou chatbot genérico (não resolve, irrita e gera mais atrito do que valor). Empresas querem atendimento que resolve de verdade, sem depender de fila humana para tudo.

A solução

Um time de 5 agentes de IA especialistas — pedidos, produtos, suporte técnico, cobrança e um orquestrador que decide quem atende — trabalhando em equipe, como um time de atendimento real. Cada agente sabe sua função; quando um não resolve sozinho, passa o bastão para o colega certo, sem o cliente repetir a história do zero.

Por que isso vende sozinho, ao vivo

- **A troca de bastão acontece na tela.** Cliente reclama de produto com defeito → agente de suporte diagnostica → transfere para o agente de produto, que já recomenda uma alternativa mais barata, sozinho, sem intervenção humana.
- **O sistema lembra do cliente.** Se ele demonstrou se importar com preço numa conversa, dias depois — em assunto totalmente diferente — a IA já prioriza opções mais baratas, sem perguntar de novo.
- **Configuração sem deploy.** Trocar a personalidade, o modelo ou a permissão de um agente é editar um documento. Desligar um agente em produção e ver o sistema se adaptar sozinho, na hora, é um momento de choque positivo em qualquer demo.
- **Isolamento de dado por identidade.** Nenhum cliente nunca vê dado de outro — segurança por identidade, não por sorte.

Onde o MongoDB brilha (mensagem de negócio)

Normalmente, coordenar múltiplos agentes de IA é a parte cara e frágil de qualquer projeto — filas externas, orquestradores adicionais, código difícil de mudar. Aqui, essa coordenação inteira — quem atendeu, para quem passou, por quê, o que a IA lembra do cliente — vive dentro do mesmo banco de dados que já guarda pedido, fatura e produto. Uma única plataforma, sem precificar quatro ferramentas diferentes: implantação mais rápida, manutenção mais barata, e um painel de auditoria que já vem de graça.

2. O pitch — linguagem técnica

Arquitetura

5 agentes (orchestrator, order_agent, product_agent, support_agent, billing_agent) coordenados por roteamento em 2 níveis: regra determinística barata (regex/keywords em ai_brain.routing_rules) resolve o caso óbvio sem custo de LLM; caso ambíguo vai para o orquestrador (Claude Haiku), que classifica a intenção e nunca responde direto ao cliente.

MongoDB como plano de coordenação, não só de dados

Collection	Papel
ai_brain.agent_registry	Persona, modelo, fallback, tools e budget por agente — editável em runtime, sem deploy.
agent_handoffs / agent_traces	Cada transferência entre agentes (máx. 2 saltos/turno) é um documento auditável. “Colaboração é u
customer_memory	Memória de longo prazo com supersessão transacional por (customer_key, fact_type). Fato extraído
semantic_cache	Isolado por agent + area + customer_key + question_norm — hit de um cliente nunca vaza para outr
products_catalog / kb_articles	\$vectorSearch com Automated Embedding (voyage-4); ranking ponderado (relevância + rating + est

Tempo real e segurança

- **Change Streams:** GET /api/events/stream (SSE) tail em agent_handoffs, filtrado por ownership; fallback poll em modo sem Atlas.
- **Ownership:** customer_key só sai do claim JWT sub, nunca do payload — todo filtro de pedido/fatura é reconstruído do zero.
- **Escrita restrita:** único write do sistema é order_agent fazendo \$set.status contra allowlist.
- **Guardrails:** mascaramento de PII antes de LLM/memória/log, guardrail de entrada e saída.
- **Resiliência:** budget de tokens por agente + global, deadline de 120s, retry só em erro transitório, fallback de modelo por agente.

Resultado: 8 padrões de arquitetura de agente (roteamento, handoff, memória, cache, segurança, observabilidade, retrieval híbrido, tempo real) sobre uma única collection store — sem Kafka, sem Redis, sem orquestrador externo.

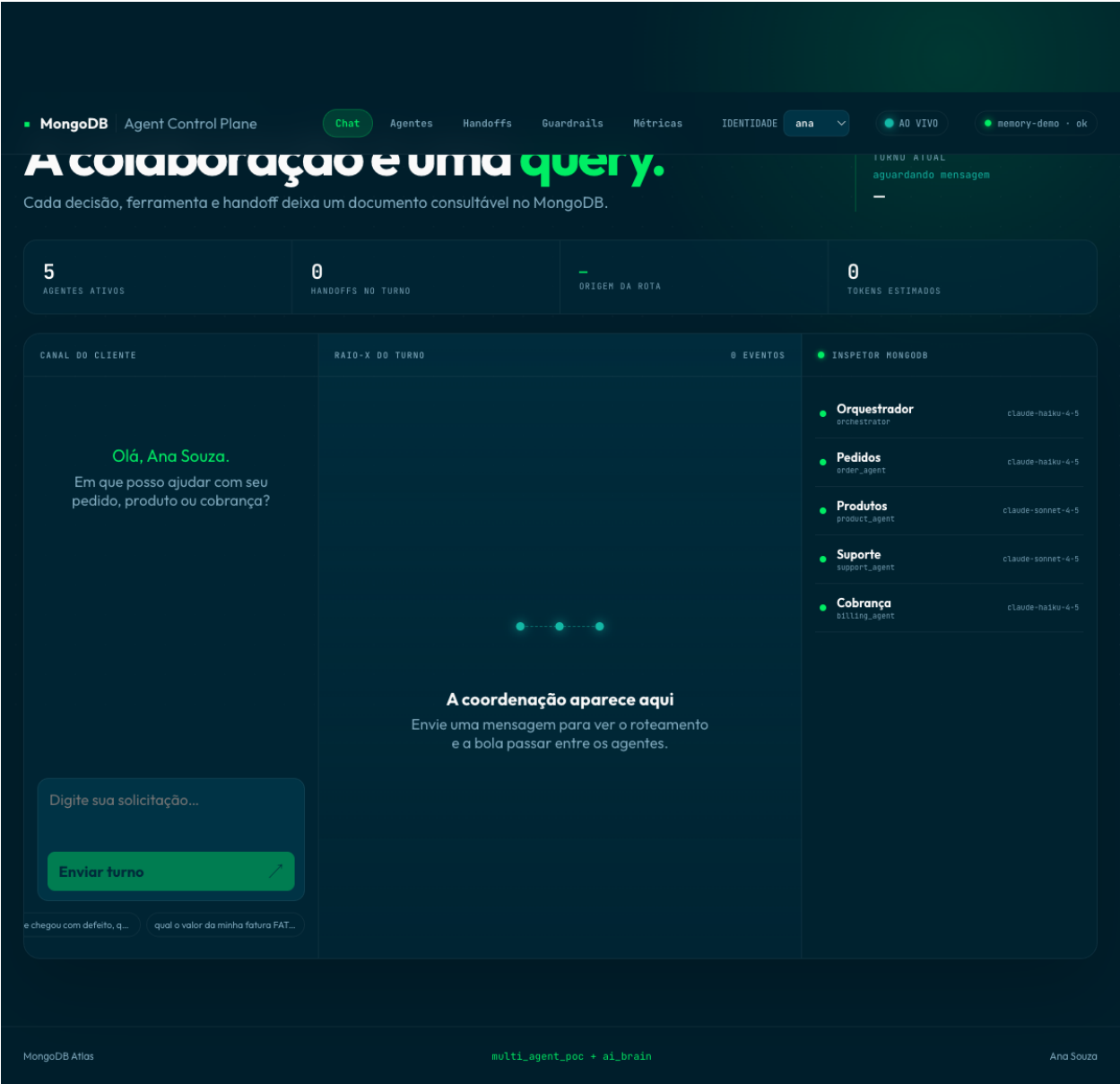
3. Roteiro de apresentação ao cliente

Passo a passo com print de tela real da aplicação e o porquê de cada clique. Use multiagent no terminal para abrir a PoV antes de começar.

Passo 0 — Abrir a aplicação

O que fazer: Tela inicial já mostra os 5 agentes ativos no painel lateral e o indicador “ao vivo” pulsando no topo — prova que o Change Stream do MongoDB já está conectado antes mesmo da primeira pergunta.

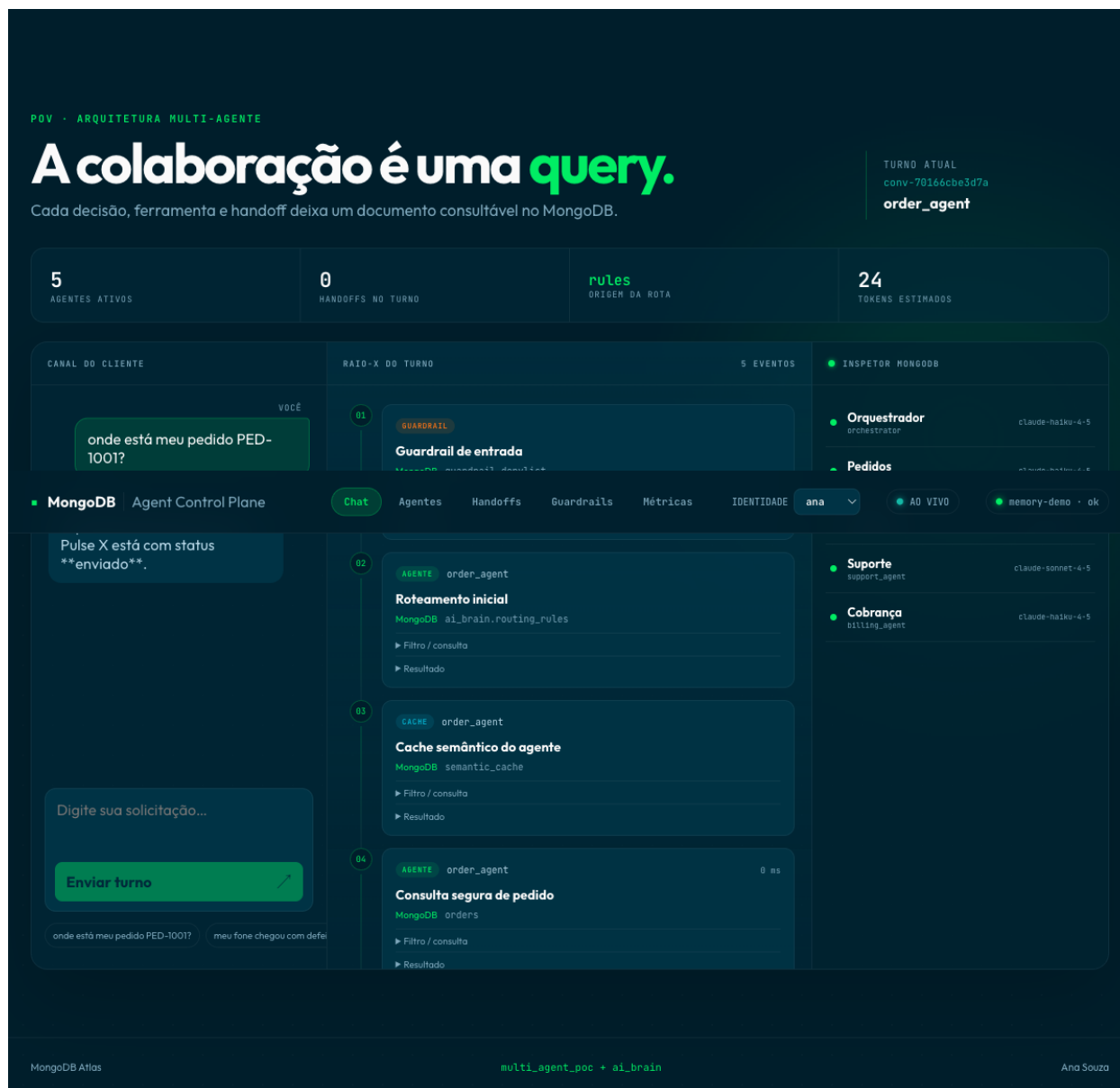
O que isso prova: Passa a mensagem de que o sistema já está “vivo” e observável, não é uma tela estática.



Passo 1 — Roteamento barato, sem custo de LLM

O que fazer: Digite ou clique no atalho: “onde está meu pedido PED-1001?”. Aponte a Timeline: o card “Roteamento inicial” mostra origem rules — nenhuma chamada de modelo foi feita para decidir quem atende.

O que isso prova: Mostra economia real: perguntas óbvias não geram custo de LLM, só perguntas ambíguas passam pelo orquestrador.



Passo 2 — Handoff entre agentes — o momento-chave da demo

O que fazer: Digite: “meu fone chegou com defeito, quero um parecido mais barato”. O `support_agent` diagnostica e transfere para o `product_agent`, que recomenda um produto mais barato sozinho. Aponte o card “Handoff explícito” na Timeline.

O que isso prova: Mostra dois agentes especializados colaborando de forma autônoma, com o motivo da transferência registrado como documento — auditável, não uma caixa preta.

POV • ARQUITETURA MULTI-AGENTE

A colaboração é uma query.

Cada decisão, ferramenta e handoff deixa um documento consultável no MongoDB.

TURNO ATUAL

conv-70166cbe3d7a

product_agent

5

AGENTES ATIVOS

1

HANDOFFS NO TURNO

orchestrator

ORIGEM DA ROTA

299

TOKENS ESTIMADOS

FEED AO VIVO • AGENT_HANDOFFS

CHANGE STREAM

• support_agent → product_agent

cliente pediu alternativa de produto após o diagnóstico

CANAL DO CLIENTE

RAIO-X DO TURNO

9 EVENTOS

INSPECTOR MONGODB

VOCE

01

GUARDRAIL

Guardrail de entrada

MongoDB customer_memory

Orquestrador

orchestrator

claude-haiku-4-5

Pedidos

product_agent

MongoDB Agent Control Plane

Chat

Agentes

Handoffs

Guardrails

Métricas

IDENTIDADE

ana

AO VIVO

memory-demo

ok

Pulse X está com status

enviado.

VOCE

meu fone chegou com defeito,

quero um parecido mais barato

PRODUCT_AGENT

A orientação da base é:

Registre evidências e solicite

troca em até sete dias após o

recebimento.

Digite sua solicitação...

Enviar turno

02

MEMORIA

Fato extraído do turno e persistido (supersessão)

MongoDB customer_memory

Filtro / consulta

Resultado

03

AGENTE

support_agent

Roteamento inicial

MongoDB ai_brain.agent_registry

Filtro / consulta

Resultado

04

CACHE

support_agent

Cache semântico do agente

MongoDB semantic_cache

Filtro / consulta

Resultado

Suporte

support_agent

claude-sonnet-4-5

Cobrança

billing_agent

claude-haiku-4-5

MongoDB Atlas

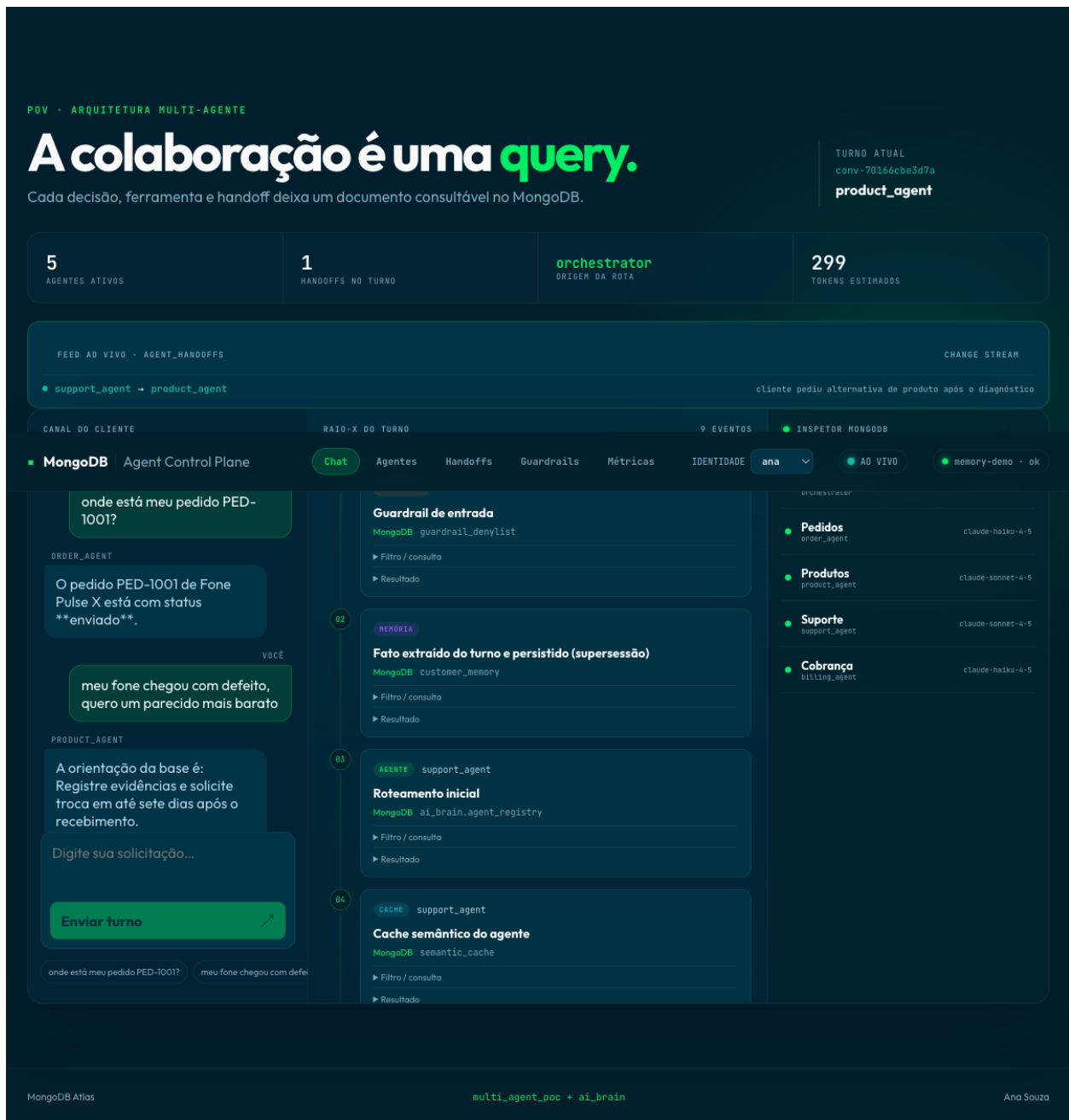
multi_agent_poc + ai_brain

Ana Souza

Passo 3 — Feed ao vivo e memória que muda o comportamento

O que fazer: Sem sair da tela, observe o painel “feed ao vivo” mostrando o handoff em tempo real (Change Stream), e repare que a resposta do product_agent já veio filtrada por preço mais baixo — o sistema lembrou que este cliente valoriza economia, mesmo sem ele repetir isso nesta pergunta.

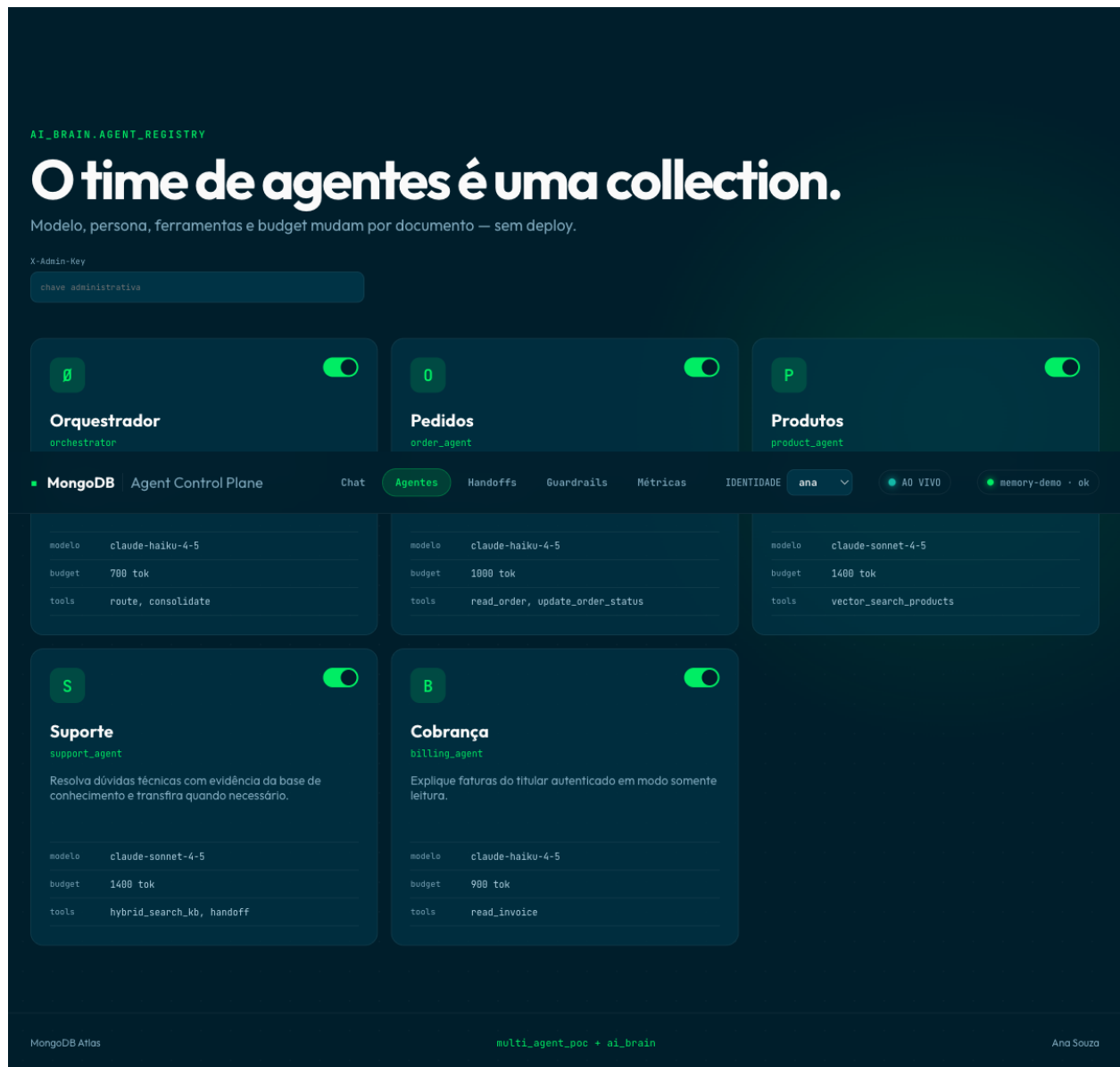
O que isso prova: Prova o loop fechado de memória (a IA aprende e aplica, não só guarda) e a coordenação em tempo real (Change Streams do Atlas), dois diferenciais que a maioria dos concorrentes não tem.



Passo 4 — Configuração sem deploy

O que fazer: Vá para a aba Agentes, informe a chave administrativa e desative o product_agent no switch. Repita a pergunta do Passo 2: o orchestrador agora usa um agente de fallback, sem reiniciar nada.

O que isso prova: Prova que o “time de agentes” é um documento editável — ajustar comportamento em produção não exige deploy, só um update.



Passos complementares (sem necessidade de print): na aba **Handoffs**, mostre a cadeia persistida como documento; troque a identidade no seletor do topo e repita uma pergunta de pedido para provar isolamento entre clientes; feche em **Métricas** mostrando tokens, cache hits e latência por agente.